NAZARBAYEV UNIVERSITY

Numerical Methods with Applications
Math 351 – 1L,2L
Fall 2026

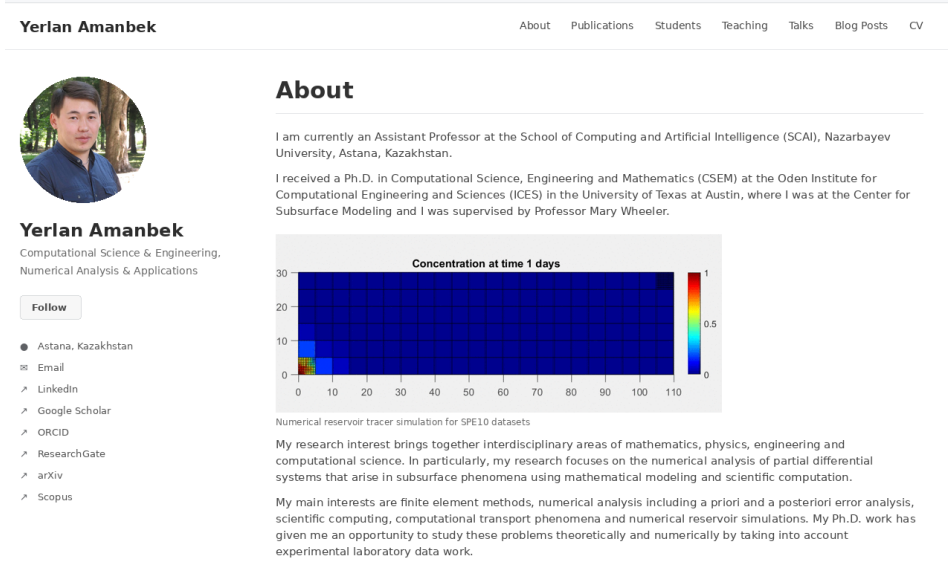
August 17, 2026

NAZARBAYEV UNIVERSITY

General Information

Professor Yerlan Amanbek

Website: https://yerlanamanbek.github.io/



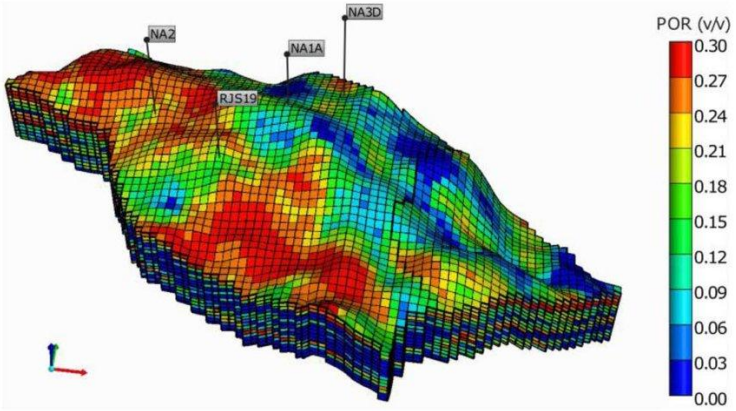
Lecture room: 7e.329

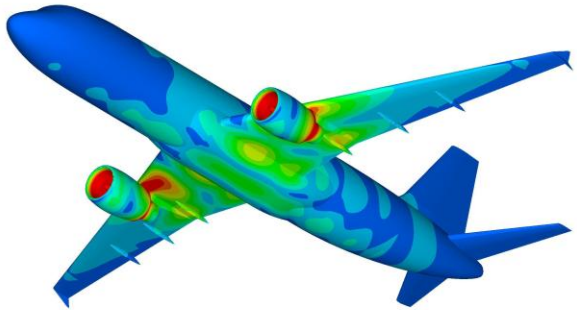
Lecture time:

1L: M W F 4:00-4:50 p.m.
2L: M W F 5:00-5:50 p.m.

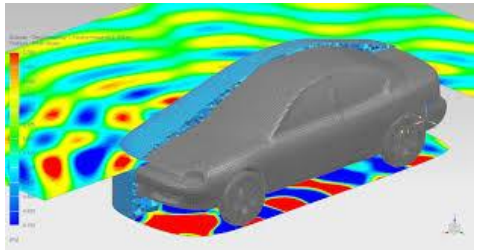


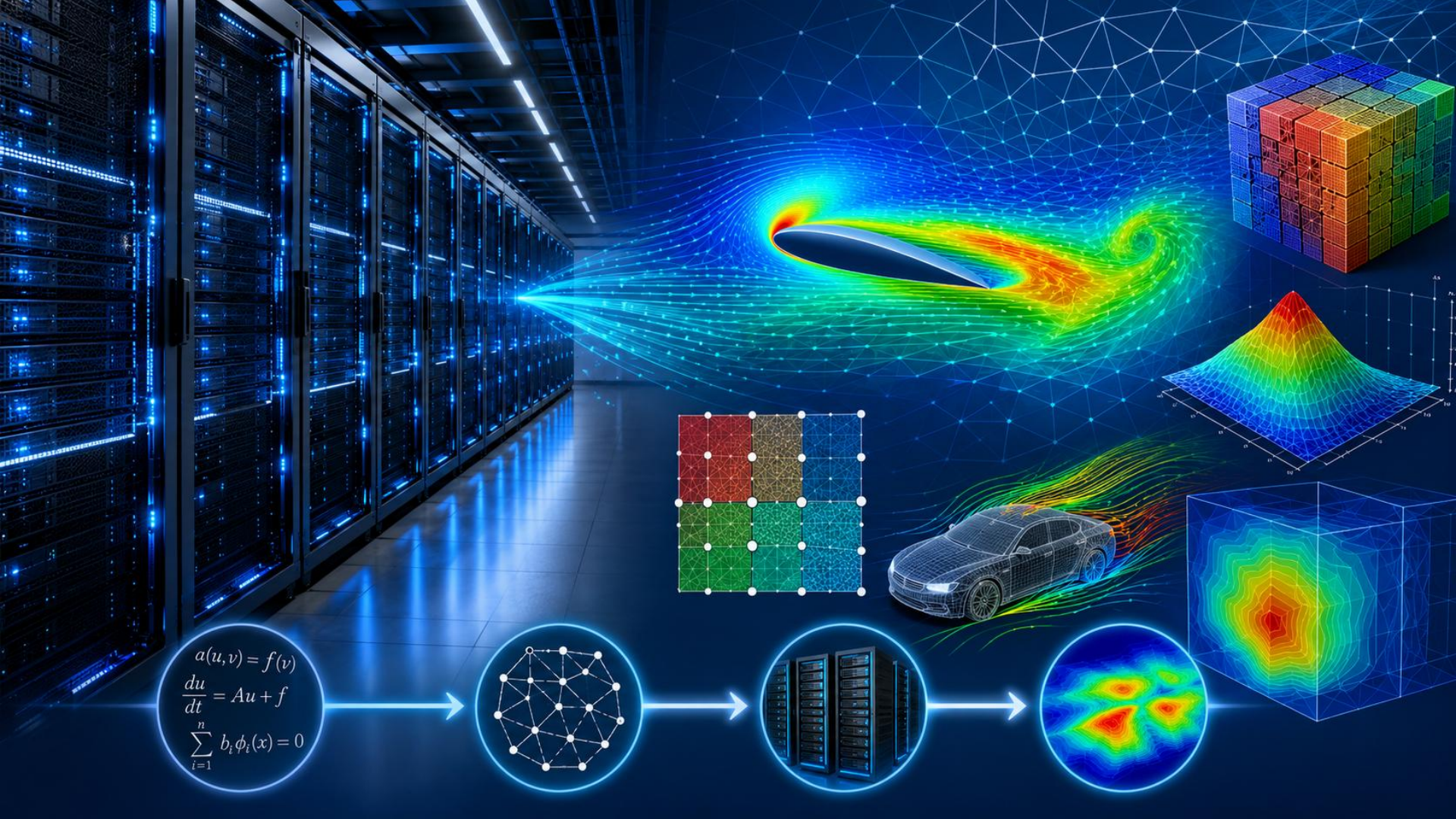


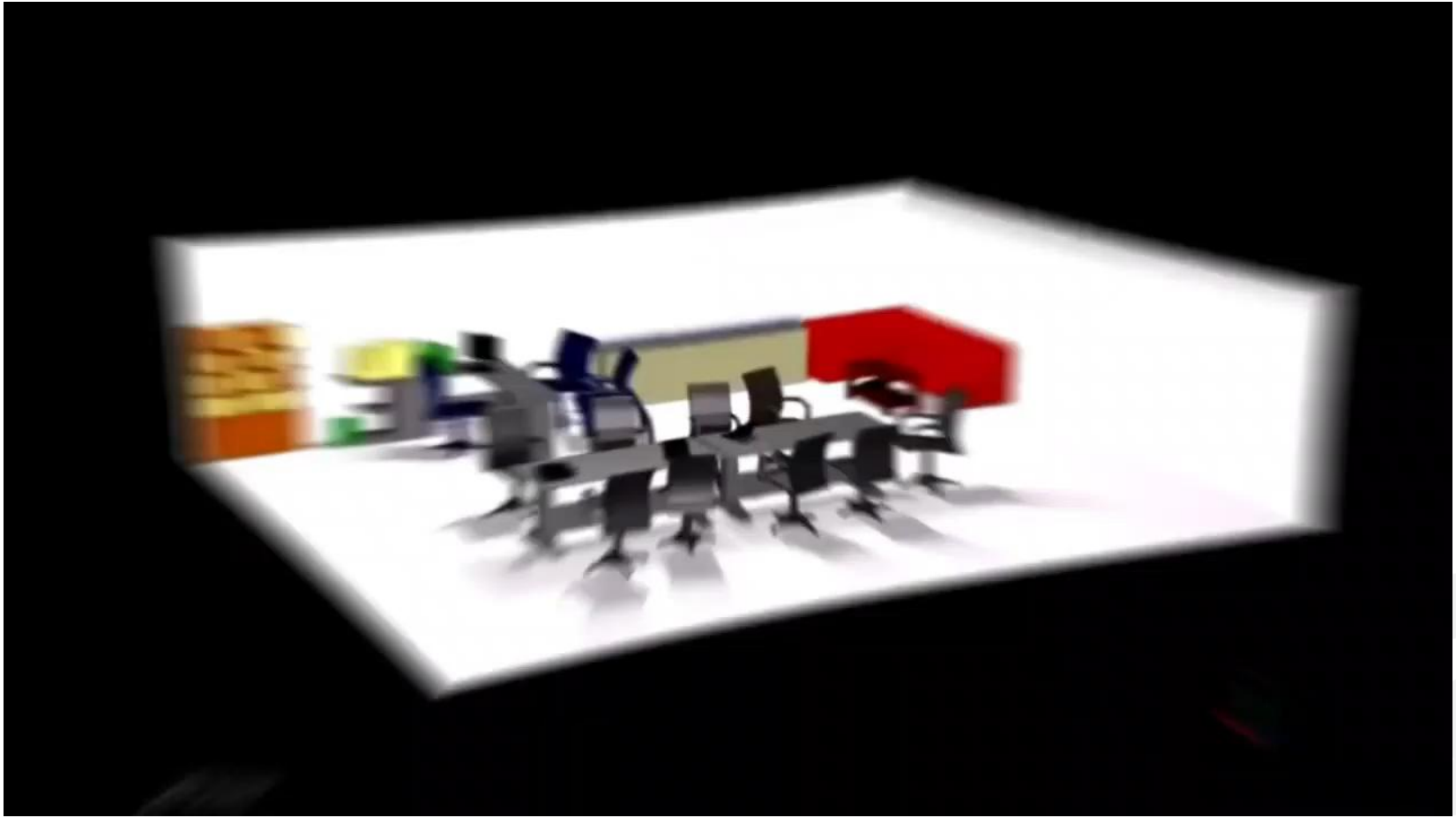




https://www.top500.org




$a(u,v) = f(v)$
$\frac{du}{dt} = Au + f$
$\sum_{i=1}^{n} b_i \phi_i(x) = 0$

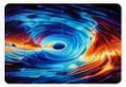


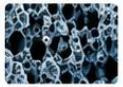

Ansys
2026R1
EMPOWER
ENGINEERING
ORGANIZATIONS



Observed Phenomenon

Physical world / Real system





Examples:

- PDEs (elliptic, parabolic, hyperbolic)
- ODEs / DAE systems
- Integral equations
- Optimization problems

Mathematical Model

Governing equations, ICs/BCs, assumptions

Goal:

- Understand behavior
- Predict / Simulate
- Optimize / Control

Discretization

Convert continuous model to a discrete form



FDM • FEM • FVM • Spectral • IGA …

Solution Algorithm

Choose numerical method to solve the discrete system



Direct • Iterative • Time stepping • Nonlinear solvers

Accuracy
Close to the true solution

Efficiency
Low computational cost

Robustness
Works reliably for a wide range of cases

Stability
Bounded errors, well-behaved solution

Implementation

Translate algorithm into code and run simulations

Programming Environment
Languages & tools
(Python, C++, MATLAB, Julia, …)

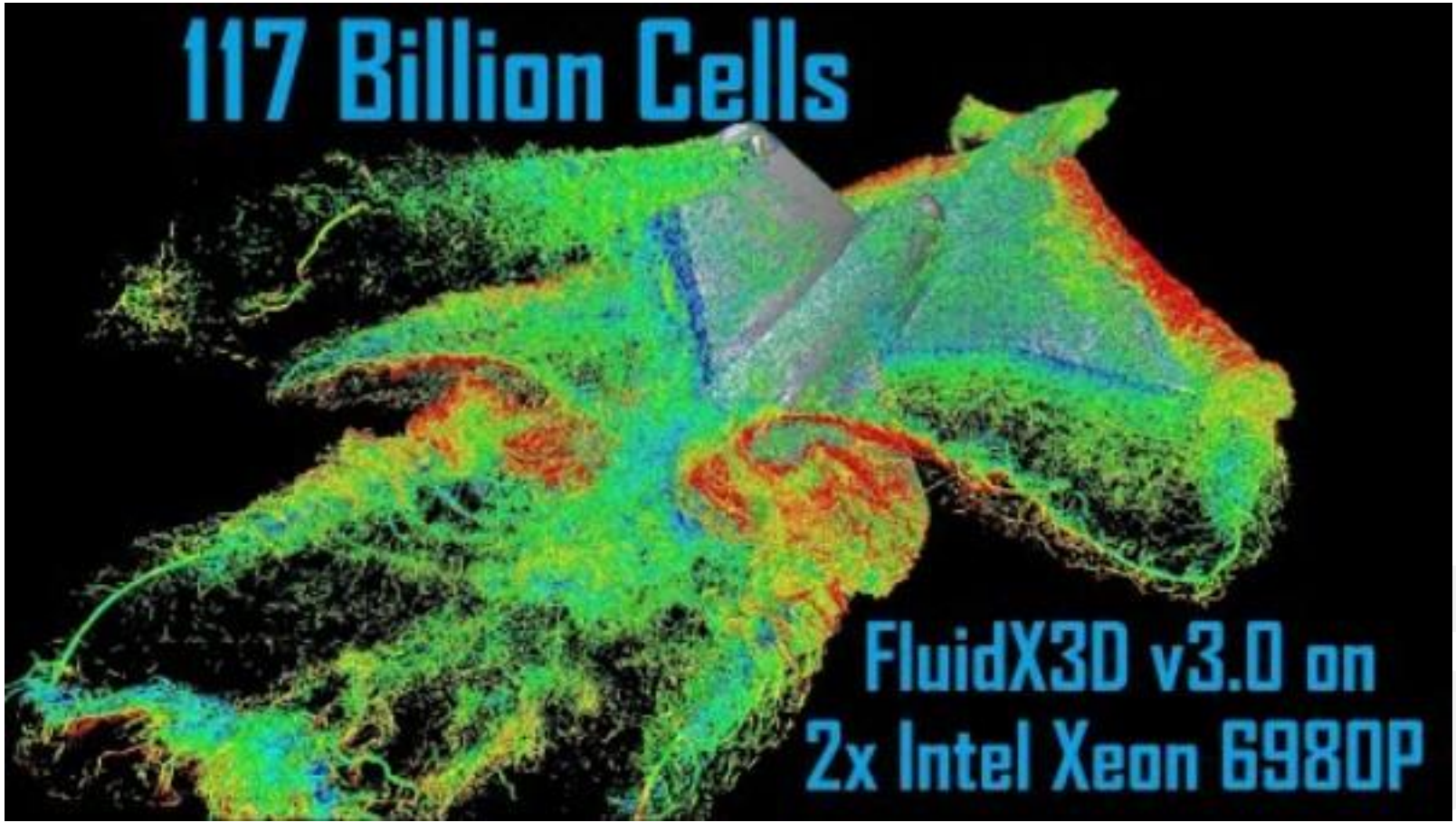
Data Structures
Vectors, matrices, sparse formats, meshes

Computing Architecture
CPU / GPU / HPC
Parallel & distributed computing

Model refinement & improvement

Scientific Computing = Mathematics + Algorithms + Software + Hardware

Turning mathematical models into reliable, efficient and accurate numerical solutions.


117 Billion Cells
FluidX3D v3.0 on
2x Intel Xeon 6980P

Methodology

- Approach the study of numerical algorithms by studying errors.
  Not really glorious, but useful.
- Problem solving environment: MATLAB

How to measure errors

- Can measure errors as absolute or relative, or a combination of both.
- The *absolute error* in v approximating u is $|u - v|$.
- The *relative error* (assuming $u \neq 0$) is $\dfrac{|u - v|}{|u|}$.

u	v	Absolute Error	Relative Error
1	0.99	0.01	0.01
1	1.01	0.01	0.01
-1.5	-1.2	0.3	0.2
100	99.99	0.01	0.0001
100	99	1	0.01

Example 1.1

The Stirling approximation

$$v = S_n = \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$$

is used to approximate $u = n! = 1 \cdot 2 \cdots n$ for large n. The formula involves the constant $e = \exp(1) = 2.7182818\ldots$. The following MATLAB script computes and displays $n!$ and S_n, as well as their absolute and relative differences, for $1 \le n \le 10$:

Example 1.1

The Stirling approximation

$$v = S_n = \sqrt{2\pi n} \cdot \left(\frac{n}{e}\right)^n$$

is used to approximate $u = n! = 1 \cdot 2 \cdots n$ for large n. The formula involves the constant $e = \exp(1) = 2.7182818\ldots$. The following MATLAB script computes and displays $n!$ and S_n, as well as their absolute and relative differences, for $1 \le n \le 10$:

The resulting output is

1	1	0.92214	0.077863	0.077863
2	2	1.919	0.080996	0.040498
3	6	5.8362	0.16379	0.027298
4	24	23.506	0.49382	0.020576
5	120	118.02	1.9808	0.016507
6	720	710.08	9.9218	0.01378
7	5040	4980.4	59.604	0.011826
8	40320	39902	417.6	0.010357
9	3.6288e+005	3.5954e+005	3343.1	0.0092128
10	3.6288e+006	3.5987e+006	30104	0.008296

Source and type of errors

- Errors in the problem to be solved
  - In the mathematical model (an approximation to reality)
  - In input data
- Approximation errors (in the numerical algorithm)
  - Discretization errors
  - Convergence errors
  - Roundoff errors

Example

Given smooth function $f(x)$, approximate derivative at some point $x = x_0$:

$$f'(x_0) \approx \frac{f(x_0 + h) - f(x_0)}{h},$$

for a small parameter value h.

Discretization error:

$$\left| f'(x_0) - \frac{f(x_0 + h) - f(x_0)}{h} \right| \approx \frac{h}{2} \left| f''(x_0) \right|.$$

Theorem: Taylor Series.
Assume that $f(x)$ has $k+1$ derivatives in an interval containing the points x_0 and x_0+h. Then

$$f(x_0+h) = f(x_0) + hf'(x_0) + \frac{h^2}{2}f''(x_0) + \cdots + \frac{h^k}{k!}f^{(k)}(x_0) + \frac{h^{k+1}}{(k+1)!}f^{(k+1)}(\xi),$$

where ξ is some point between x_0 and x_0+h.

Big-O and Θ Notation

Throughout this text we consider various computational errors depending on a discretization step size $h > 0$ and ask how they decrease as h decreases. In other instances, such as when estimating the efficiency of a particular algorithm, we are interested in a bound on the work estimate as a parameter n increases unboundedly (e.g., $n = 1/h$).

For an error e depending on h we denote

$$e = \mathcal{O}(h^q)$$

if there are two positive constants q and C such that

$$|e| \le Ch^q$$

for all $h > 0$ small enough.

Similarly, for $w = w(n)$ the expression

$$w = \mathcal{O}(n \log n)$$

means that there is a constant $C > 0$ such that

$$w \le Cn \log n$$

as $n \to \infty$. It will be easy to figure out from the context which of these two meanings is the relevant one.

The Θ notation signifies a stronger relation than the $\mathcal{O}$ notation: a function $\phi(h)$ for small h (resp., $\phi(n)$ for large n) is $\Theta(\psi(h))$ (resp., $\Theta(\psi(n))$) if ϕ is asymptotically bounded *both above and below* by ψ.

Results

Try for $f(x) = \sin(x)$ at $x_0 = 1.2$.
(So we are approximating $\cos(1.2) = 0.362357754476674...$.)

h	Absolute error
0.1	4.716676e-2
0.01	4.666196e-3
0.001	4.660799e-4
1.e-4	4.660256e-5
1.e-7	4.619326e-8

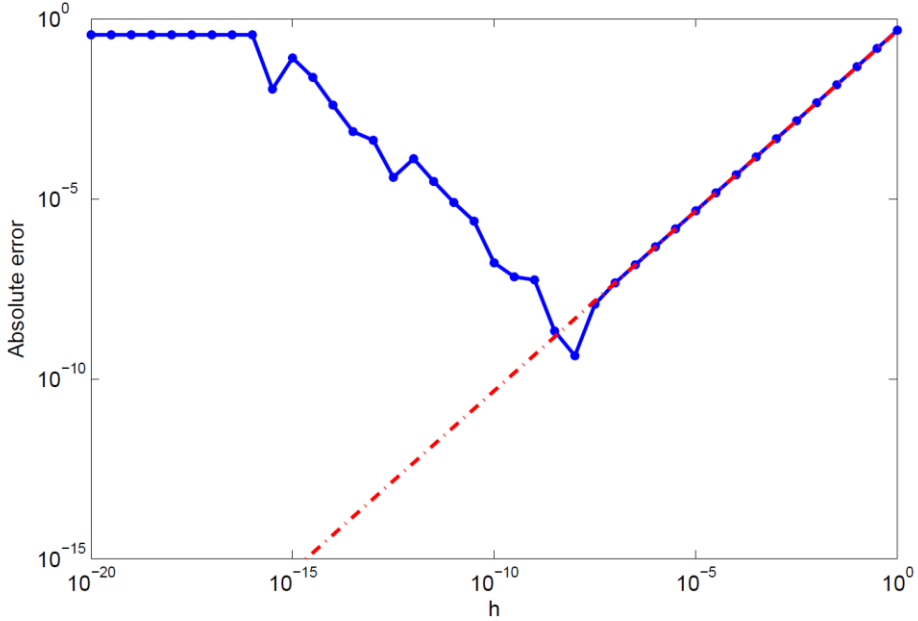
These results reflect the discretization error as expected.

Results for smaller h

h	Absolute error
1.e-8	4.361050e-10
1.e-9	5.594726e-8
1.e-10	1.669696e-7
1.e-11	7.938531e-6
1.e-13	6.851746e-4
1.e-15	8.173146e-2
1.e-16	3.623578e-1

These results reflect both discretization and roundoff errors.

Results for all h



The solid curve interpolates the computed values of $|f'(x_0) - \frac{f(x_0+h)-f(x_0)}{h}|$ for $f(x) = \sin(x),\ x_0 = 1.2$. Shown in dash-dot style is a straight line depicting the discretization error without roundoff error.

Roundoff Error

Exact number vs stored number

Exact real number

$$\pi = 3.14159265358979323846264338327 9\ldots$$

Stored (finite-precision) number

3.141593



Computers store only a finite number of digits.

Roundoff error arises because a computer stores an approximation of the exact real number.

Algorithm Properties



1. Accuracy

The degree to which the computed solution approximates the true (exact) solution.

$$\text{Error} = |\,\text{computed result} - \text{true result}\,|$$

➜ **Higher accuracy** means smaller error.



2. Efficiency (surprisingly hard to measure)

The computational resources required to obtain a solution.

- CPU time (or wall-clock time)
- Memory usage
- Number of operations (e.g., function evaluations, iterations)

➜ **More efficient** algorithms achieve the desired accuracy with less cost.



3. Robustness

The ability of the algorithm to handle a wide range of inputs and conditions reliably.

- Insensitive to small perturbations in the data
- Resistant to roundoff errors and numerical instabilities
- Works well for a variety of problem settings

➜ **More robust** algorithms are less likely to fail or produce wildly incorrect results.



Key takeaway: A good algorithm balances **accuracy**, **efficiency**, and **robustness** for the problem at hand.

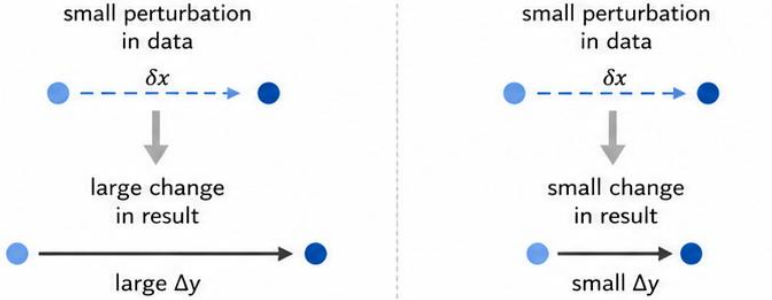
Problem conditioning and algorithm stability

Qualitatively speaking:

1 Problem conditioning

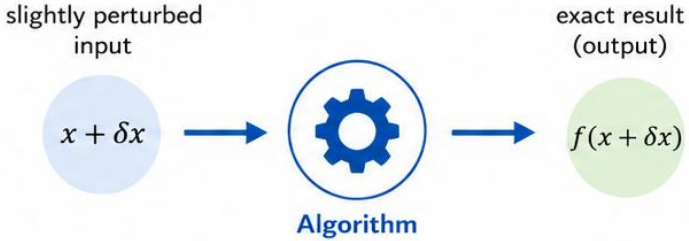
The problem is **ill-conditioned** if a small perturbation in the data may produce a large difference in the result.

The problem is **well-conditioned** otherwise.



2 Algorithm stability

The algorithm is **stable** if its output is the exact result of a slightly perturbed input.



Next, we'll see how bad **roundoff error accumulation** can be when an unstable algorithm is used.

Problem conditioning and algorithm stability

Conditioning of the problem $y = g(x)$

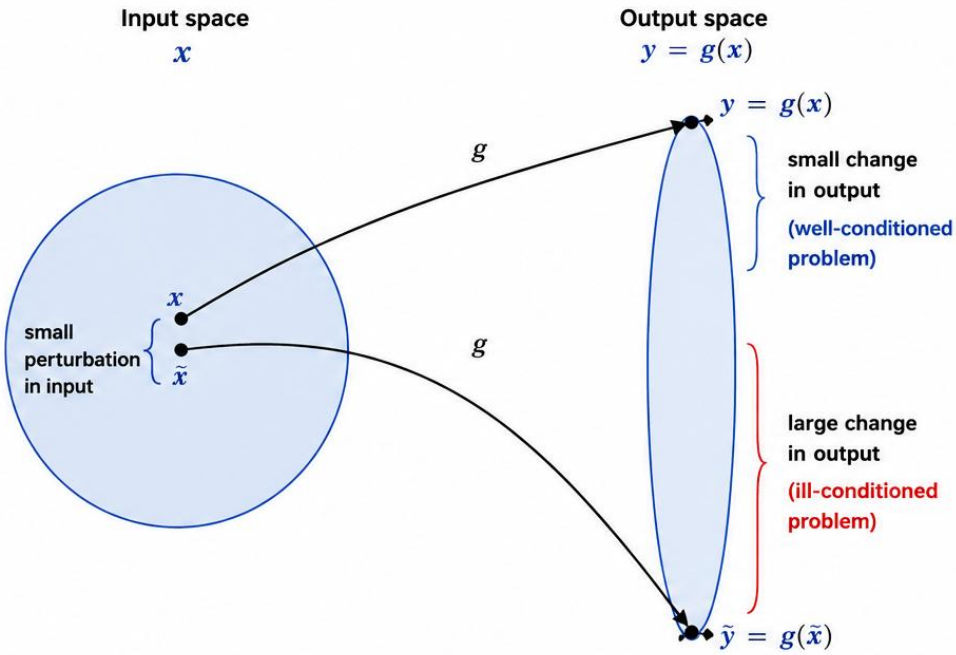
- Small changes in the input x lead to small changes in the true output $g(x)$.
  → **Well-conditioned**

- Small changes in the input x may lead to large changes in the true output $g(x)$.
  → **Ill-conditioned**

Stability of the algorithm

- A stable algorithm produces an output that is the exact result of a slightly perturbed input.
  → **Stable**

- An unstable algorithm can amplify tiny roundoff errors and produce a very different result.
  → **Unstable**



Takeaway: Even if the problem is well-conditioned, an unstable algorithm can yield a completely wrong answer due to the amplification of roundoff errors.

Unstable algorithm: an extreme example

1. Problem statement

Evaluate the integrals

$$y_n = \int_0^1 \frac{x^n}{x+10}\,dx$$

for $n = 1, 2, \ldots, 30$.

2. Analytical observation

Observe that analytically

$$y_n + 10y_{n-1} = \int_0^1 \frac{x^n + 10x^{n-1}}{x+10}\,dx = \int_0^1 x^{n-1}\,dx = \frac{1}{n}.$$

Also

$$y_0 = \int_0^1 \frac{1}{x+10}\,dx = \ln(11) - \ln(10).$$

3. Algorithm

- Evaluate $y_0 = \ln(11) - \ln(10)$.
- For $n = 1, \ldots, 30$, evaluate $y_n = \frac{1}{n} - 10\,y_{n-1}$.

Why the Algorithm is Unstable

Key idea: In the recurrence $y_n = \frac{1}{n} - 10y_{n-1}$, any small roundoff error is **multiplied by 10** at every step.

1 Problem

Evaluate

$$y_n = \int_0^1 \frac{x^n}{x+10}\,dx, \qquad n = 1, 2, \ldots, 30.$$

2 Analytical relations

From

$$y_n + 10y_{n-1} = \int_0^1 \frac{x^n + 10x^{n-1}}{x+10}\,dx = \int_0^1 x^{n-1}\,dx = \frac{1}{n}$$

Recurrence (used in the algorithm):

$$y_n = \frac{1}{n} - 10y_{n-1}$$

Also

$$y_0 = \int_0^1 \frac{1}{x+10}\,dx = \ln(11) - \ln(10)$$

3 Algorithm

- Evaluate $y_0 = \ln(11) - \ln(10)$.
- For $n = 1, \ldots, 30$, evaluate $y_n = \frac{1}{n} - 10y_{n-1}$.

4 Why is it unstable?

Let $\tilde{y}_{n-1} = y_{n-1} + e_{n-1}$ be the computed value, where e_{n-1} is the roundoff error.

Then

$$\tilde{y}_n = \frac{1}{n} - 10\tilde{y}_{n-1} = \frac{1}{n} - 10(y_{n-1} + e_{n-1})$$

Since $y_n = \frac{1}{n} - 10y_{n-1}$, the error satisfies

$$e_n = \tilde{y}_n - y_n = -10e_{n-1}$$

$$\Rightarrow \quad |e_n| = 10\,|e_{n-1}| \approx 10^n\,|e_0|$$

The error is multiplied by 10 (and its sign changes) at every step → **exponential error growth**.

5 Numerical effect (double precision)

n	0	5	10	15	20	25	30
True $y_n = \frac{1}{n}$	0.095310	0.200000	0.100000	0.066667	0.050000	0.040000	0.033333
Computed y_n	0.095310	0.200000	0.100000	0.096769	-9.17×10^{3}	-9.17×10^{7}	-9.17×10^{13}

- Even tiny initial error (e.g., $|e_0| \approx 10^{-16}$) becomes $|e_{16}| \approx 1$, $|e_{30}| \approx 10^{14}$.
- The computed values quickly blow up and lose all accuracy.

Takeaway

The forward recurrence is numerically unstable because errors are **amplified by 10** each step.

A stable alternative is the backward recurrence $y_{n-1} = \frac{1/n - y_n}{10}$, which **divides** the error by 10.

Roundoff error accumulation

1. In general, if E_n is the error after n elementary operations, then linear roundoff error accumulation is unavoidable.

   Acceptable (linear growth) $\quad E_n \simeq c_0 n E_0$

   E_n : error after n operations
   E_0 : initial error
   c_0 : constant independent of n

2. Will not tolerate **exponential** error growth such as

   Unacceptable (exponential growth) $\quad E_n \simeq c_1^n E_0$

   for some constant $c_1 > 1$

   ➜ This indicates an **unstable algorithm**.

3. In some situations, an individual error contribution can be particularly large and can sometimes be reduced.

NAZARBAYEV
UNIVERSITY

